1. **Write a python program to compute • Central Tendency Measures: Mean, Median,Mode • Measure of Dispersion: Variance, Standard Deviation**

Understanding Basic Statistics

Statistical analysis often involves understanding the central tendency and the dispersion of data. Below are the definitions, along with the formulas used for these measures.

1. Central Tendency Measures

- **Mean (μ):** The arithmetic average of a set of values. It is calculated by summing all the data points and dividing by the number of data points.

$$\text{Mean}(\mu) = \frac{\sum_{i=1}^{n} x_i}{n}$$

- **Median:** The median is the middle value in a sorted list of numbers. If the number of observations is even, the median is the average of the two middle numbers.

$$\text{Median} = \begin{cases} \text{Middle value}, & \text{if } n \text{ is odd} \\ \frac{x_{n/2}+x_{(n/2)+1}}{2}, & \text{if } n \text{ is even} \end{cases}$$

where $x_{n/2}$ and $x_{(n/2)+1}$ are the middle elements after sorting the data.

- **Mode:** The mode is the value that appears most frequently in a dataset.
  Mode = most frequent value in the dataset

2. Measures of Dispersion

- **Variance (σ^2):** Variance measures the spread of the data points around the mean. It is calculated as the average of the squared differences between each data point and the mean.

$$\text{Variance}(\sigma^2) = \frac{\sum_{i=1}^{n}(x_i - \mu)^2}{n-1}$$

where x_i are the individual data points, μ is the mean of the data, and n is the total number of data points.

- **Standard Deviation (σ):** The standard deviation is the square root of the variance, providing a measure of the average distance from the mean. It is in the same units as the data, making it more interpretable compared to variance.

$$\text{Standard Deviation}(\sigma) = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \mu)^2}{n-1}}$$

These measures provide insights into the distribution, central point, and spread of data, helping to understand the data's overall behaviour.

Code:

```
# Prompt user to enter data
user_input = input("Enter numbers separated by commas: ")

# Convert the input string to a list of integers
numbers = [int(num) for num in user_input.split(',')]

# Calculate the number of elements
count = len(numbers)

# Calculate the mean (average)
sum_of_numbers = sum(numbers)
mean_value = sum_of_numbers / count

# Sort the list for median calculation
sorted_numbers = sorted(numbers)

# Calculate the median
if count % 2 == 0:
    median_value = (sorted_numbers[count // 2 - 1] + sorted_numbers[count // 2]) / 2
else:
    median_value = sorted_numbers[count // 2]

# Calculate the mode
```

```
frequency_dict = {}
highest_frequency = 0

for num in numbers:
    if num in frequency_dict:
        frequency_dict[num] += 1
    else:
        frequency_dict[num] = 1
    if frequency_dict[num] > highest_frequency:
        highest_frequency = frequency_dict[num]

# Get all numbers with the highest frequency
modes = [num for num, freq in frequency_dict.items() if freq == highest_frequency]

# Calculate variance
squared_diff_sum = sum((num - mean_value) ** 2 for num in numbers)
variance_value = squared_diff_sum / count

# Calculate standard deviation
std_deviation = variance_value ** 0.5

# Display results
print(f"Mean = {mean_value}")
print(f"Median = {median_value}")
print(f"Mode = {modes if len(modes) < count else 'No mode (all values are unique)'}")
print(f"Variance = {variance_value}")
print(f"Standard Deviation = {std_deviation}")
```

output:

Sample Run1:

```
$ python3 Stat_Measures.py
Enter numbers separated by commas: 1,2,3,4,4,5,6
```

Mean = 3.5714285714285716

Median = 4

Mode = [4]

Variance = 2.5306122448979593

Standard Deviation = 1.5907898179514348

Week2:) Study of Python Basic Libraries such as Statistics, Math, Numpy and Scipy

1. Statistics Library:

The statistics module provides functions to perform basic statistical operations like calculating the mean, median, mode, variance, and standard deviation.

Key Functions:

- **mean(data):** Returns the arithmetic mean of the data.
- **median(data):** Returns the median value.
- **mode(data):** Returns the mode (most frequent value).
- **variance(data):** Returns the variance of the data.
- **stdev(data):** Returns the standard deviation.

```
import statistics
# Input data from user
data=input("Enter data separated by comma:")
data=[int(x) for x in data.split(',')]
# Compute Mean
mean=statistics.mean(data)
# Compute Median
median=statistics.median(data)
# Compute Mode
mode=statistics.mode(data)
# Compute Standard Deviation
std_dev=statistics.stdev(data)
# Compute Variance
variance=statistics.variance(data)
# Output results
print(f"Mean={mean}")
print(f"Median={median}")
print(f"Mode={mode}")
print(f"Standard Deviation={std_dev}")
print(f"Variance={variance}")
```

Sample Run:

$python3 Stat_Lib.py.py

Enter data separated by comma:1,2,3,4,4,5,6

Mean=3.5714285714285716

Median=4

Mode=4

Standard Deviation=1.7182493859684491

Variance=2.9523809523809526

2. Math Library:

The math module offers a range of mathematical functions that operate on numbers and perform tasks such as trigonometric calculations, logarithms, and square roots.

Key Functions:

- **math.ceil(x):** Returns the smallest integer greater than or equal to x.
- **math.factorial(x):** Returns the factorial of a non-negative integer x.
- **math.exp(x):** Returns exe^xex, where eee is the base of natural logarithms (approximately 2.718).
- **math.sqrt(x):** Returns the square root of x.
- **math.floor(x):** Returns the largest integer less than or equal to x.
- **math.gcd(x, y):** Returns the greatest common divisor (GCD) of two integers x and y.
- **math.pow(x, y):** Returns xyx^yxy (x raised to the power of y).
- **math.radians(angle):** Converts an angle from degrees to radians.
- **math.sin(x):** Returns the sine of x, where x is in radians.
- **math.cos(x):** Returns the cosine of x, where x is in radians.
- **math.tan(x):** Returns the tangent of x, where x is in radians

```
import math
x=int(input("Enter a number:"))
result1=math.ceil(x)
result2=math.factorial(x)
result3=math.exp(x)
result4=math.sqrt(x)
result5=math.floor(x)
n1=int(input("Enter number1:"))
n2=int(input("Enter number2:"))
result6=math.gcd(n1,n2)
result7=math.pow(n1,n2)
angle=int(input("Enter an angle:"))
a=math.radians(angle)
```

```
result8=math.sin(a)
result9=math.cos(a)
result10=math.tan(a)
print(f"Using math.ceil({x}): {result1}")
print(f"Using math.factorial({x}):{result2}")
print(f"Using math.exp({x}):{result3}")
print(f"Using math.sqrt({x}):{result4}")
print(f"Using math.floor({x}):{result5}")
print(f"Using math.gcd({n1},{n2}):{result6}")
print(f"Using math.pow({n1},{n2}):{result7}")
print(f"Using math.sin({angle}):{result8}")
print(f"Using math.cos({angle}):{result9}")
print(f"Using math.tan({angle}):{result10:.15f}")
```

Output:

Sample Run:

$python3 Math_Lib.py.py

Enter a number:4

Enter number1:3

Enter number2:2

Enter an angle:1

Using math.ceil(4): 4

Using math.factorial(4):24

Using math.exp(4):54.598150033144236

Using math.sqrt(4):2.0

Using math.floor(4):4

Using math.gcd(3,2):1

Using math.pow(3,2):9.0

Using math.sin(1):0.01745240643728351

Using math.cos(1):0.9998476951563913

Using math.tan(1):0.017455064928218

3. Numpy Library:

numpy is a library for numerical computing in Python. It is used for working with arrays and matrices, along with a collection of mathematical functions to operate on these data structures

code:

```
import numpy as np

# Get number of rows and columns from user

rows=int(input("Enter number of rows:"))

cols=int(input("Enter number of columns:"))

# Get array elements from user

elements=list(map(float, input(f"Enter {rows*cols} elements separated by spaces:").split()))

# Create an array from user input

user_array=np.array(elements).reshape(rows,cols)

# Create an array of ones and zeros

ones_array=np.ones((rows, cols))

zeros_array=np.zeros((rows, cols))

# Transpose the user array

transposed_array=np.transpose(user_array)

# Reshape the user array (to a flat array and back to the original shape for demonstration)

reshaped_array = user_array.reshape(rows * cols)

min_value=np.min(elements)

max_value=np.max(elements)

# Display the results

print("User Array:\n",user_array)

print("Ones Array:\n",ones_array)

print("Zeros Array:\n",zeros_array)

print("Transposed Array:\n",transposed_array)

print("Reshaped Array (flattened):\n",reshaped_array)

print("Minimum Value:",min_value)

print("Maximum Value:",max_value)
```

output:

Sample Run:

```
$python3 Numpy_Lib.py

Enter number of rows:2

Enter number of columns:2

Enter 4 elements separated by spaces:1 2 3 4

User Array:

 [[1. 2.]

 [3. 4.]]

Ones Array:

 [[1. 1.]

 [1. 1.]]

Zeros Array:

 [[0. 0.]

 [0. 0.]]

Transposed Array:

 [[1. 3.]

 [2. 4.]]

Reshaped Array (flattened):

 [1. 2. 3. 4.]

Minimum Value: 1.0

Maximum Value: 4.0
```

4. Scipy Library:

Scipy builds on numpy and provides a wide range of advanced mathematical, scientific, and engineering functions, including optimization, integration, interpolation, and statistical operations.

Code:

```
from scipy import linalg

import numpy as np

from scipy import special

from scipy import constants

# Print constants

print("Seconds in a minute:",constants.minute) # 60.0

print("Seconds in an hour:",constants.hour) # 3600.0
```

```
print("Length of an inch in meters:",constants.inch) # 0.0254
print("Volume of a liter in cubic meters:",constants.liter) # 0.001
# Dynamic input for exponentiation
exp_input = float(input("Enter the exponent for 10^x:"))
print(f"10^{exp_input} =",special.exp10(exp_input))
# Dynamic input for angles in degrees
angle=float(input("Enter an angle for sine and cosine(in degrees):"))
#Calculate sine and cosine of the input angles
print(f"sin({angle} degrees):",special.sindg(angle))
print(f"cos({angle} degrees):",special.cosdg(angle))
# Dynamic input for matrix
rows = int(input("Enter the number of rows for the matrix:"))
cols = int(input("Enter the number of columns for the matrix:"))
mat = list(map(float, input(f"Enter {rows * cols} elements separated by spaces:").split()))
# Convert the flat list into a 2D NumPy array
mat=np.array(mat).reshape((rows, cols))
# Calculate the determinant if the matrix is square
if rows == cols:
        x = linalg.det(mat)
        print(f'Determinant of the matrix\n{mat}\n is: {x}')
else:
        print("Determinant can only be calculated for square matrices.")
```

output:

Sample Run:

$python Scipy_Lib.py

Seconds in a minute: 60.0

Seconds in an hour: 3600.0

Length of an inch in meters: 0.0254

Volume of a liter in cubic meters: 0.001

Enter the exponent for 10^x:2

10^2.0 = 100.0

Enter an angle for sine and cosine(in degrees):90

sin(90.0 degrees): 1.0

cos(90.0 degrees): -0.0

Enter the number of rows for the matrix:2

Enter the number of columns for the matrix:2

Enter 4 elements separated by spaces:2 3 4 5

Determinant of the matrix

[[2. 3.]

 [4. 5.]]

 is: -2.0000000000000004

week3: **Study of Python Libraries for ML application such as Pandas and Matplotlib**

1. Pandas

Pandas is a fast, powerful, and flexible library used for data analysis and manipulation. It provides data structures like DataFrames and Series that are widely used in ML for managing structured data.

Here u need take students dataset

code

```
import pandas

df=pandas.read_csv('students.csv')

print(df)

print(df.loc[[0,1]])

print(df.head())

print(df.tail())

print(df.isnull())

print(df.info())
```

output:

Sample Run:

$python3 Pandas_Lib.py

```
   Student_NO     Name Branch Year  Contact_NO
```

0 1201 Raghu IT III 1234

1 1202 Sai IT III 1234

2 1203 Sravan IT III 1234

3 1204 Karthik IT III 1234

4 1205 Gaanesh IT III 1234

5 1206 Vamshi IT III 1234

6 1207 Shiva IT III 1234

7 1208 Racha IT III 1234

8 1209 Abu IT III 1234

9 1210 Nithish IT III 1234

Student_NO Name Branch Year Contact_NO

0 1201 Raghu IT III 1234

1 1202 Sai IT III 1234

Student_NO Name Branch Year Contact_NO

0 1201 Raghu IT III 1234

1 1202 Sai IT III 1234

2 1203 Sravan IT III 1234

3 1204 Karthik IT III 1234

4 1205 Gaanesh IT III 1234

Student_NO Name Branch Year Contact_NO

5 1206 Vamshi IT III 1234

6 1207 Shiva IT III 1234

7 1208 Racha IT III 1234

8 1209 Abu IT III 1234

9 1210 Nithish IT III 1234

Student_NO Name Branch Year Contact_NO

0 False False False False False

1 False False False False False

2 False False False False False

3 False False False False False

4 False False False False False

5 False False False False False

6 False False False False False

7 False False False False False

8 False False False False False

9 False False False False False

RangeIndex: 10 entries, 0 to 9

Data columns (total 5 columns):

 # Column Non-Null Count Dtype

--- ------ -------------- -----

0 Student_NO 10 non-null int64

1 Name 10 non-null object

2 Branch 10 non-null object

3 Year 10 non-null object

4 Contact_NO 10 non-null int64

dtypes: int64(2), object(3)

memory usage: 532.0+ bytes

None

2.Matplotlib:

Matplotlib is a plotting library used for creating static, interactive, and animated visualizations. It is commonly used in ML for data exploration and model evaluation through plots

```
import matplotlib.pyplot as plt

import numpy as np

# Taking user inputs for x and y coordinates

x_input = input("Enter the x coordinates separated by spaces: ")

y_input = input("Enter the y coordinates separated by spaces: ")

# Splitting the input string into a list of strings, then converting them to float

x = np.array([float(i) for i in x_input.split()])

y = np.array([float(i) for i in y_input.split()])

# Checking if the number of x and y coordinates match

if len(x) != len(y):
```

```
        print("Error: The number of x and y coordinates must be the same.")
else:
        plt.plot(x, y, marker='o') # Plot with markers at data points
        plt.xlabel('X-axis')
        plt.ylabel('Y-axis')
        plt.title('User Input Plot')
        plt.grid(True) # Adding a grid for better visualization
        plt.show()
```

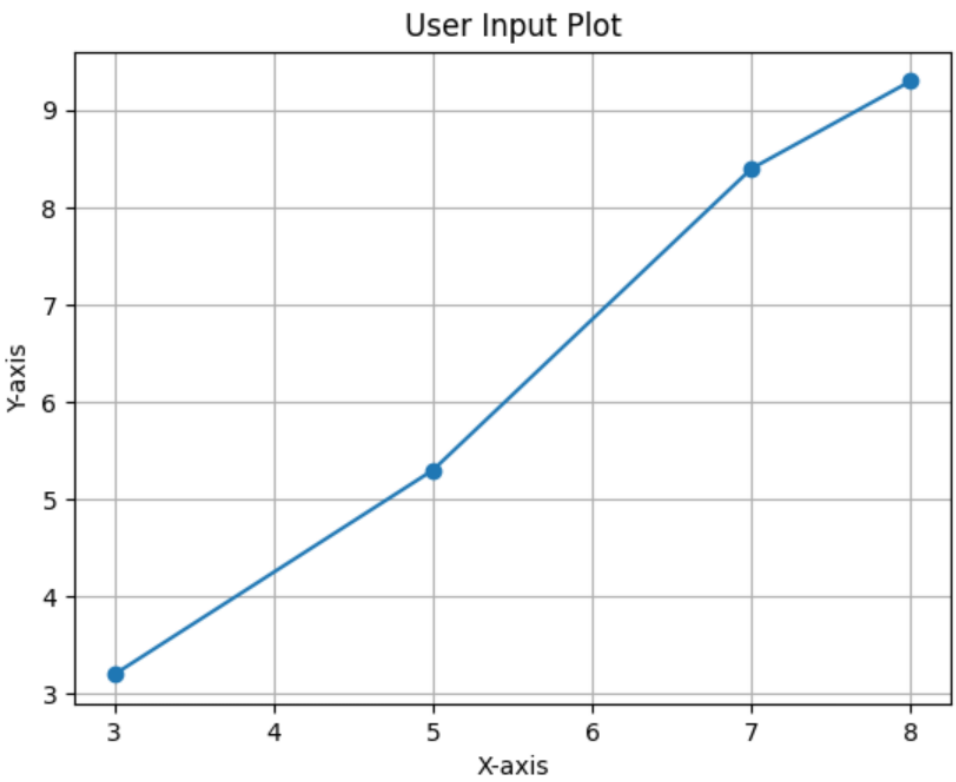
output:

Sample Run:

$python3 Matplot_Lib.py

Enter the x coordinates separated by spaces: 3 5 7 8

Enter the y coordinates separated by spaces: 3.2 5.3 8.4 9.3



Week4: Write a Python program to implement Simple Linear Regression

Simple linear regression is the simplest form of linear regression and it involves only one independent variable and one dependent variable.

$y = \beta 0 + \beta 1 X$

where:

y is the dependent variable

X is the independent variable

β0 is the intercept

β1 is the slope

here u need to take salary dataset

code:

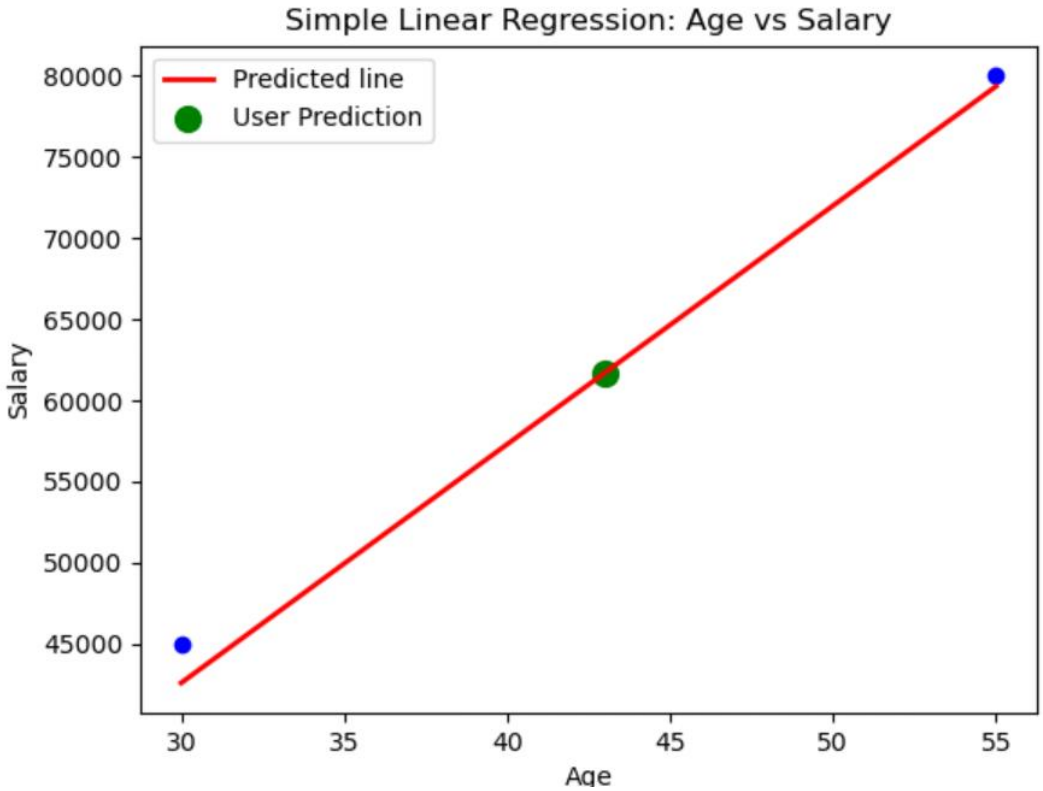
```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score


#Load the data from the CSV file

data = pd.read_csv('salary_data.csv')

x= data[['age']] # Independent variable

y = data['salary'] # Dependent variable


#Split the data into training and testing sets

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,random_state=1)


model=LinearRegression()

model.fit(x_train, y_train)


y_pred=model.predict(x_test)
```

```
#Model coefficients
print(f"Intercept (a0): {model.intercept_}")
print(f"Slope (a1): {model.coef_}")
r2=r2_score(y_test, y_pred)

#Print evaluation metrics
print (f"R-squared Score: ",r2)

#User input for age
user_age = float(input("Enter age to predict salary: "))

#Predict salary for the given age
predicted_salary = model.predict(pd.DataFrame([[user_age]], columns=['age']))
print (f"The predicted salary for age {user_age} is:",predicted_salary)
plt.scatter (x_test, y_test, color='blue')
plt.plot(x_test, y_pred, color='red', linewidth=2, label='Predicted line')
plt.scatter (user_age, predicted_salary, color='green', s=100, label='User Prediction')
plt.xlabel('Age')
plt.ylabel('Salary')
plt.title('Simple Linear Regression: Age vs Salary')
plt.legend()
plt.show()
```

output

Sample Run:

```
$ python3 Linear_Regression.py
Intercept (a0): -1522.5102319235884
Slope (a1): [1470.66848568]
R-squared Score:  0.9899167950543547
Enter age to predict salary: 43
The predicted salary for age 43.0 is: [61716.23465211]
```



Week5: Implementation of Multiple Linear Regression for House Price Prediction using sklearn

Multiple Linear Regression attempts to model the relationship between two or more features and a response by fitting a linear equation to observed data. The steps to perform multiple linear Regression are almost similar to that of simple linear Regression. The Difference Lies in the evaluation. We can use it to find out which factor has the highest impact on the predicted output and how different variables relate to each other.

Y = b0 + b1 * x1 + b2 * x2 + b3 * x3 + …… bn * xn

Y = Dependent variable

x1, x2, x3, …… xn = multiple independent variables

here u need take house price dataset

code:

```
import pandas as pd

import numpy as np
```

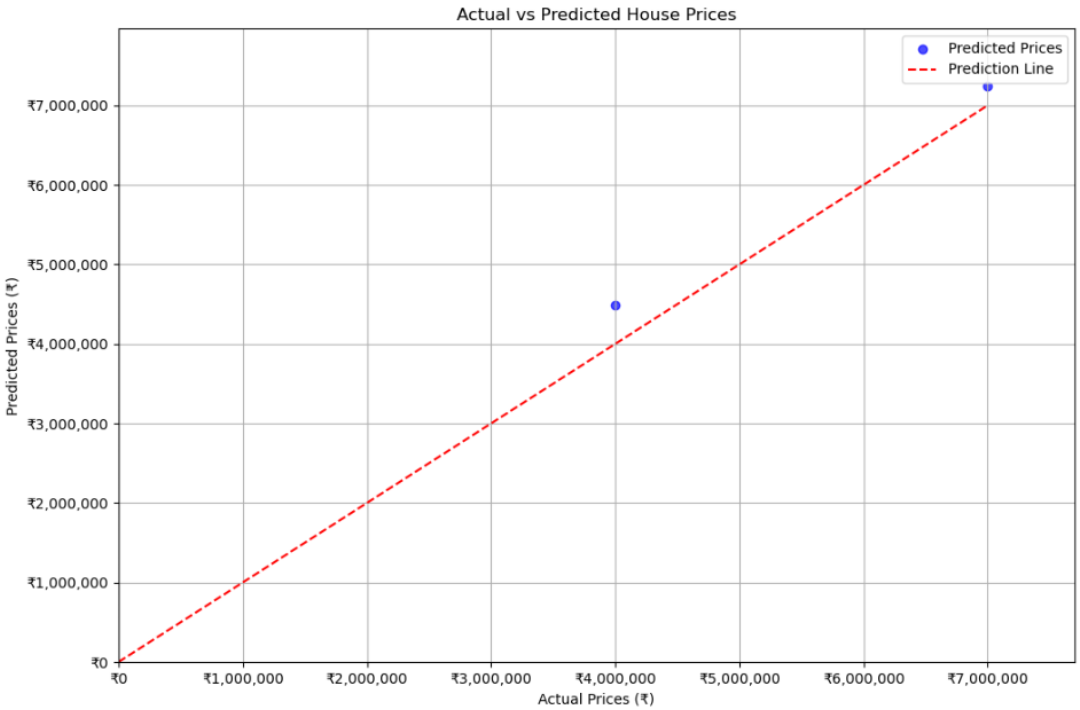
```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from matplotlib.ticker import FuncFormatter

data = pd.read_csv('house_data.csv')
X = data.drop('price', axis=1)
y = data['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_state=42)
model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}")
print(f"R^2 Score: {r2_score(y_test, y_pred):.2f}")
print(f"Model Coefficients: {model.coef_}")
print(f"Model Intercept: {model.intercept_}")

area = float(input("Enter area (sq ft): "))
bedrooms = int(input("Enter number of bedrooms: "))
floors = int(input("Enter number of floors: "))
age = int(input("Enter age of the house: "))
input_data = pd.DataFrame([[area, bedrooms, floors, age]], columns=['area', 'bedrooms', 'floors',
'age'])
input_data_encoded = input_data.reindex(columns=X.columns, fill_value=0)
predicted_price = model.predict(input_data_encoded)

print(f"Predicted price for the house: ₹{predicted_price[0]:,.2f}")
plt.figure(figsize=(12, 8)) # Adjusted size
plt.scatter(y_test, y_pred, alpha=0.7, label="Predicted Prices", color='blue', marker='o')
```

```
plt.plot([0, max(y_test)], [0, max(y_test)], color='red', linestyle='--', label="Prediction Line")
plt.xlabel("Actual Prices (₹)")
plt.ylabel("Predicted Prices (₹)")
plt.title("Actual vs Predicted House Prices")

# Use safe formatter instead of set_xticklabels/set_yticklabels
formatter = FuncFormatter(lambda x, pos: f'₹{int(x):,}')
plt.gca().xaxis.set_major_formatter(formatter)
plt.gca().yaxis.set_major_formatter(formatter)

plt.legend()
plt.xlim(0, max(y_test) * 1.1)
plt.ylim(0, max(y_pred) * 1.1)
# Grid and display
plt.grid(True)
plt.show()
```

output

Sample Run:

```
$ python3 Multiple_Linear_Regression.py
Mean Squared Error: 147133574968.89
R^2 Score: 0.93
Model Coefficients: [  2518.89168766 198012.87433529 198012.87433529  94458.43828715]
Model Intercept: -818499.8600628739
Enter area (sq ft): 200
Enter number of bedrooms: 3
Enter number of floors: 2
Enter age of the house: 2
Predicted price for the house: ₹864,259.73
```


Actual vs Predicted House Prices
Predicted Prices
Prediction Line
₹7,000,000
₹6,000,000
₹5,000,000
₹4,000,000
₹3,000,000
₹2,000,000
₹1,000,000
₹0
Predicted Prices (₹)
₹0
₹1,000,000
₹2,000,000
₹3,000,000
₹4,000,000
₹5,000,000
₹6,000,000
₹7,000,000
Actual Prices (₹)